A Project Report

On

Agentic AI Procurement Solution

Submitted in partial fulfilment of the requirements of

University of Mumbai

For the Degree of

Bachelor of Engineering

in

COMPUTER ENGINEERING

Submitted by

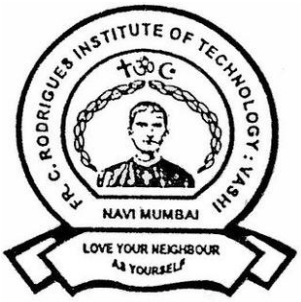
More Anjali Dinesh
Sankaye Vaidehi Jagdeep
Sawant Niraj Ravindra
Shete Anooj

Supervised by

Dr. Jyoti More

&

Mr. Alok
(External guide)



Department of Computer Engineering
Fr. Conceicao Rodrigues Institute of Technology
An Autonomous Institute & Permanently Affiliated to University of Mumbai
Sector 9A, Vashi, Navi Mumbai - 400703

UNIVERSITY OF MUMBAI

APPROVAL SHEET

This is to certify that the project entitled

AGENTIC AI PROCUREMENT SOLUTION

Submitted by

More Anjali Dinesh	**1023207**
Sankaye Vaidehi Jagdeep	**1023239**
Sawant Niraj Ravindra	**1023240**
Shete Anooj	**1023250**

Supervisor / Project Guide	**Project Coordinator**
Internal Examiner	**External Examiner**
Head of Department	**Principal**

COLLEGE SEAL

Date : _____

Place : _____

Declaration

We declare that this written submission for B.E. Declaration entitled "**Agentic AI Procurement Solution**" represents our ideas in our own words and where others' ideas or words have been included. We have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any ideas/data/fact/source in our submission. We understand that any violation of the above will cause disciplinary action by the institute and also evoke penal action from the sources which have thus not been properly cited or from whom permission has not been taken when needed.

Project Group Members:

1. More Anjali Dinesh, 1023207

_____

2. Sankaye Vaidehi Jagdeep, 1023239

_____

3. Sawant Niraj Ravindra, 1023240

_____

4. Shete Anooj, 1023250

_____

Abstract

IT procurement covers the purchase, licensing, renewal and risk assessment of hardware, software and SaaS services. In many organisations it is still handled through e-mail, spreadsheets and separate tools, which leads to slow approvals, duplicate payments, unused licences, missed contract renewals and weak visibility of vendor risk. Existing techniques such as OCR, rule-based extraction, approval workflows and machine learning usually address one stage of the procurement cycle in isolation.

This report presents an Agentic AI-based IT Procurement Intelligence Platform built as an event-driven system of four cooperating agents: Document and Vendor Intelligence, Approval and Inventory Intelligence, Contract and Risk Analysis, and a Platform agent that provides authentication, business rules, notifications and a conversational assistant. Each agent is an independent FastAPI service that reacts to events on a Kafka-compatible bus (Redpanda), decides with rules or trained models, and publishes its result through a transactional outbox; long-running steps run as Temporal workflows, and decisions that are uncertain or sensitive are handed to a person.

Uploads are scanned with ClamAV and read with pdfplumber, Docling or PaddleOCR. A TF-IDF and logistic-regression classifier, regular-expression field extraction and an optional LayoutLMv3 cross-check produce a confidence-scored record, and records below a confidence of 0.8 go to a review queue. Vendors are matched by normalised fuzzy name, onboarding validates GSTIN check digits and IFSC codes, bank details require a second person's verification, and duplicate invoices are flagged. Requests follow spend-tier approval chains with SLA escalation and Redis-locked stock reservation; an Isolation Forest with SHAP explanations proposes licence reclaims after a grace period. Contracts are generated from templates, signed in the application, and read by a Legal-BERT model fine-tuned on CUAD, and vendors carry a Random Forest risk score with hard overrides for a cancelled GSTIN or a sanctions match, monitored for drift with the Population Stability Index.

On synthetic data, the shipped classifier reached a macro-F1 of 1.0 against 0.912 for a keyword baseline; Legal-BERT reached F1 scores of 0.85, 0.80 and 0.91 on three clause types of the CUAD test split, against 0.50, 0.19 and 0.02 for rules; the licence-anomaly model reached a ROC-AUC of 0.954; and the risk model reached a ROC-AUC of 0.715 against a ceiling of 0.739 set by its label noise. Field extraction was complete on text PDFs but reached only 49% on photographed images. The implementation passes 823 service unit tests and four end-to-end suites through the gateway. All datasets are synthetic, so these figures compare methods rather than predict performance on an organisation's real documents.

Keywords: agentic AI, IT procurement, event-driven architecture, document intelligence, human-in-the-loop, vendor risk, anomaly detection, contract analysis.

Contents

Abstract iii

List of Figures vi

List of Tables vii

List of Listings viii

1 Introduction 1
- 1.1 Background . . . 2
- 1.2 Motivation . . . 2
- 1.3 Aim and Objectives . . . 2
  - 1.3.1 Aim . . . 2
  - 1.3.2 Objectives . . . 2
- 1.4 Problem Statement . . . 3
- 1.5 Scope . . . 3
- 1.6 Report Outline . . . 4

2 Study of the System 5
- 2.1 About the Technique . . . 6
- 2.2 Various Available Techniques . . . 7
  - 2.2.1 Document Intelligence . . . 7
  - 2.2.2 Document Classification . . . 7
  - 2.2.3 Vendor Deduplication and Identity Verification . . . 7
  - 2.2.4 Workflow Orchestration and Concurrency Control . . . 8
  - 2.2.5 Usage Anomaly Detection and Explainability . . . 8
  - 2.2.6 Contract Analysis and Clause Extraction . . . 8
  - 2.2.7 Vendor Risk Scoring and Drift Monitoring . . . 8
  - 2.2.8 Multi-Agent and Agentic Architectures . . . 9
- 2.3 Related Works . . . 9
- 2.4 Literature Summary and Research Gap . . . 11

3 Proposed System 12
- 3.1 Problem Statement . . . 13
- 3.2 Scope . . . 13
- 3.3 Scope Design . . . 14
- 3.4 Proposed System . . . 14
  - 3.4.1 Overview and Architecture . . . 14
  - 3.4.2 The Four Agents . . . 15
  - 3.4.3 Document Processing Pipeline . . . 17

3.4.4 Inter-Agent Communication and Workflow . . . 18
3.4.5 Use Case View . . . 19
3.4.6 Technology Stack . . . 20
3.4.7 Key Design Principles . . . 20

4 Design of the System **23**
4.1 Requirement Engineering . . . 24
4.1.1 Requirement Elicitation . . . 24
4.1.2 Software Lifecycle Model . . . 24
4.1.3 Requirement Analysis . . . 25
4.1.3.1 Functional Requirements . . . 25
4.1.3.2 Non-Functional Requirements . . . 27
4.1.4 UML Diagrams and Data Flow Diagrams . . . 27
4.1.4.1 Use Case Diagram . . . 27
4.1.4.2 Data Flow Diagram – Level 0 . . . 28
4.1.4.3 Data Flow Diagram – Level 1 . . . 28
4.1.4.4 Sequence Diagram . . . 29
4.1.4.5 Activity Diagram: Document Processing . . . 29
4.1.4.6 Purchase-Request State Machine . . . 29
4.1.4.7 Entity Relationship Diagram . . . 29
4.1.5 Cost Analysis . . . 33
4.1.6 Hardware and Software Requirements . . . 33
4.2 System Architecture . . . 33
4.2.1 UI/UX Design . . . 35
4.2.2 Block Diagram . . . 35
4.2.3 Main Data Paths . . . 35
4.3 x . . . 38

References **39**

Acknowledgement **40**

Appendix A: Timeline Chart **41**

Appendix B: Sustainable Development Goal (SDG) Mapping **41**

Appendix C: Complex Engineering Problem Identification and Evaluation **42**

List of Figures

3.1 Overall architecture of the proposed system 15
3.2 Document processing pipeline of the Document and Vendor Intelligence agent 17
3.3 Main event flows between the agents 18
3.4 Use case diagram of the IT procurement platform 20

4.1 Iterative and incremental development cycle 25
4.2 Level 0 data flow diagram (context diagram) 28
4.3 Level 1 data flow diagram 28
4.4 Sequence diagram of the purchase-request journey 30
4.5 Activity diagram of document processing 31
4.6 State machine of a purchase request 32
4.7 Entity relationship diagram of the main entities 32
4.8 Block diagram of the deployed system 36

List of Tables

2.1 Traditional procurement practices and their shortcomings 6
2.2 Comparison of related works 10
2.3 Comparison of related works (continued) 10
2.4 Techniques reviewed and their use in the proposed system 11

3.1 Problems in the current procurement process and their impact 13
3.2 Summary of the four agents in the proposed system 16
3.3 Event topics exchanged between the agents 18
3.4 Technology stack of the proposed system 21

4.1 Functional requirements 25
4.2 Non-functional requirements 27
4.3 Level 1 DFD processes 29
4.4 Indicative cost analysis 33
4.5 Hardware requirements 34
4.6 Software requirements 34
4.7 Principal screens and their purposes 36
4.8 Architectural layers 37

List of Listings

Chapter 1

Introduction

1.1 Background

IT procurement is an important part of an organisation's operations because it covers the acquisition and management of hardware, software licences, SaaS subscriptions and related services. The process runs through several stages: identifying a vendor, processing its documents, obtaining approval, managing inventory, handling the contract and assessing vendor risk. In many organisations these activities are spread across e-mail, spreadsheets, invoices and separate systems, which makes it hard to keep a consistent, current view of procurement. Approvals are delayed, purchases are duplicated, licences go unused, contracts are difficult to track and vendor-related risks grow unnoticed. Automation and artificial intelligence offer a way to make these activities faster and more reliable.

1.2 Motivation

The project is motivated by the time spent on manual procurement work and the lack of transparency and control that results from it. Employees verify documents, review vendor details, chase approvals, check stock and licence utilisation, and keep track of contract renewals. These tasks become error-prone as volumes grow, and some of the resulting errors, such as paying a fraudulent change of bank details, are costly. A system that automates the routine steps, links the stages of the procurement cycle and asks a person only when a decision is uncertain or sensitive would be of clear benefit. The project therefore implements a set of special-purpose software agents for document processing, vendor matching, approvals, inventory and licence analysis, contract management and risk detection.

1.3 Aim and Objectives

1.3.1 Aim

To develop an Agentic AI-based IT Procurement Intelligence Platform that integrates, automates and intelligently manages the major activities of the IT procurement lifecycle while keeping human oversight of critical decisions.

1.3.2 Objectives

- To process procurement documents such as purchase orders, invoices and quotations, and extract their key fields with a confidence score for each result.
- To match vendors against existing records, validate their tax and bank identifiers, and detect duplicate vendors and duplicate invoices.
- To automate approval workflows based on spend tiers, an approval-authority matrix, SLA deadlines and inventory availability, with escalation when an approver does not respond.
- To detect under-used software licences with an anomaly-detection model and propose their reclamation under human control.

- To support contract management by generating contracts, collecting signatures, extracting renewal-related clauses and issuing renewal reminders.
- To assess vendor risk with a machine learning model, explain each score, and monitor the model for drift.
- To provide shared platform services: authentication, role-based access control, configurable business rules, notifications, monitoring and audit logging.
- To keep humans involved in low-confidence results and in critical decisions such as changes to vendor payment details.

1.4 Problem Statement

IT procurement involves collecting procurement documents, verifying vendor information, processing invoices and quotations, obtaining approvals, managing software licences, monitoring contracts and assessing vendor risk. These activities are often handled through several systems and manual processes, which causes delays, duplicate records, wasted resources and limited visibility across the procurement lifecycle.

Traditional procurement systems mainly record transactions; they do little to analyse procurement data or to support decisions. Identifying duplicate vendors or invoices, detecting under-used licences, tracking renewals and assessing vendor risk therefore take significant manual effort. At the same time, automated decisions made without a measure of confidence and without human oversight can lead to incorrect procurement actions.

There is therefore a need for an integrated solution that automates routine procurement activities, analyses the relevant data, supports approval decisions and monitors procurement risk. The proposed platform addresses this need by combining document processing, vendor intelligence, workflow automation, licence management, contract monitoring, risk scoring and human-in-the-loop controls in one system.

1.5 Scope

The platform covers the following activities of the IT procurement lifecycle:

- **Procurement document processing:** receiving documents by upload, e-mail or a watched folder; scanning, parsing, classifying and extracting structured fields from purchase orders, invoices and quotations.
- **Vendor intelligence:** matching extracted vendor names against existing records, onboarding vendors with GSTIN and IFSC validation, and holding new or changed bank details for verification by a second user.
- **Invoice checks:** flagging likely duplicate invoices and matching invoices line by line against the purchase request.
- **Approval workflow automation:** routing purchase requests through spend-tier approval chains with SLA timers, reminders, escalation and inventory reservation.
- **Software licence management:** scoring licence usage for anomalies and creating reclaim requests that a person approves after a grace period.

- **Contract management:** generating contracts from templates, signing them in the application, extracting renewal type, notice period and end date, and scheduling renewal reminders.
- **Vendor risk assessment:** scoring vendors with a machine learning model and hard-override rules, listing the contributing factors, and monitoring the model for drift.
- **Shared platform services:** authentication, role-based access control, business rules editable at run time, e-mail notifications, audit logging, metrics, centralised logs and a conversational assistant.
- **Human-in-the-loop decision making:** routing low-confidence documents, payment-detail changes, reclaims and high-risk signatures to authorised users.

The platform is a decision-support and workflow-automation system; it does not replace human decision-makers in critical procurement activities. Payment execution, ERP integration and production deployment are outside the scope (Section 3.2).

1.6 Report Outline

This report is organised into six chapters. Chapter 1, **Introduction**, presents the background, motivation, aim and objectives, problem statement and scope. Chapter 2, **Study of the System**, reviews the techniques relevant to each part of the system and compares related research. Chapter 3, **Proposed System**, describes the problem, the scope and the proposed platform: its architecture, agents, document pipeline, event flow and technology stack. Chapter 4, **Design of the System**, covers requirement engineering, the lifecycle model, UML diagrams and data-flow diagrams, cost analysis, hardware and software requirements, and the system architecture. Chapter **??**, **Result and Discussion**, presents screenshots of the implemented system, its main logic, the testing performed and the evaluation of its models. Chapter **??**, **Conclusion and Future Scope**, summarises the outcomes, limitations and possible extensions. The report ends with references and appendices covering the project timeline, SDG mapping, complex engineering problem evaluation and project outcomes.

Chapter 2

Study of the System

Procurement departments handle a steady stream of documents, including purchase orders, invoices and vendor quotations, and these rarely arrive in the same form twice. Some come by e-mail, some through vendor portals and others are scans or photographs of printed pages. In many organisations someone still reads each one and types the details into a system. Contracts are followed up in spreadsheets, vendors are vetted once at onboarding, and software licences are checked with a simple inactivity rule. Over time this leads to typing mistakes, duplicate payments, vendors whose identity was never confirmed, contracts that renew without anyone noticing, and licences that are paid for but never used. Fraud is a further concern: in Business Email Compromise (BEC), an attacker poses as a supplier and asks for bank details to be changed, and the request succeeds because nobody verifies it independently.

This chapter reviews the techniques behind each part of the proposed platform. Because each agent relies on a different set of methods, the review covers a range of techniques rather than a single one. It begins with how procurement is traditionally done and where that approach falls short, reviews the main techniques for each sub-problem together with the choice made in this project, compares ten related research works, and ends with the research gap the project addresses.

2.1 About the Technique

Traditional procurement processing rests on four habits. The first is manual or template-based data entry: staff key in invoice and purchase-order details, or an OCR template is built for each vendor's layout. The second is static approval rules, where a request follows a fixed chain with no deadline, no escalation and no awareness of remaining stock. The third is periodic vendor review, which in practice often means one questionnaire at onboarding. The fourth is the flat-threshold licence audit, in which a seat is flagged when nobody has logged in for a fixed number of days. These habits work at small volumes but break down as an organisation grows; Table 2.1 summarises where each struggles.

Table 2.1: Traditional procurement practices and their shortcomings

Area	**Traditional approach**	**Shortcoming**
Data capture	Manual entry and per-vendor templates	Slow and error-prone; templates break whenever a vendor changes its layout.
Vendor verification	Manual lookup and e-mail confirmation	Vulnerable to BEC fraud; duplicate vendor records accumulate.
Approvals	E-mail chains and fixed rules	No SLA timers or escalation; overlapping requests can claim the same stock.
Licence management	Fixed inactivity threshold	Ignores usage patterns and wrongly flags seasonal or weekend dips.
Contract tracking	Calendar reminders	Notice periods are missed, resulting in unintended automatic renewals.
Risk assessment	One-time questionnaire	Becomes outdated; misses later events such as a cancelled tax registration or a sanctions listing.

These gaps motivate an automated, event-driven approach. The approach taken here is not to hand every decision to a model. Pretrained components handle perception-heavy work such as reading page layouts and contract language; small models are trained where a task is specific to this project (document classification, licence anomalies and vendor risk), and each trained model is kept only if it beats a simple baseline on held-out data;

and plain rules remain wherever certainty and auditability matter more than flexibility, such as spend-tier routing, sanctions overrides and dual control of bank details.

2.2 Various Available Techniques

2.2.1 Document Intelligence

Template and regular-expression approaches are fast and predictable but brittle when vendors change layouts. OCR engines such as Tesseract [**?**] and PaddleOCR [**?**] recover text from scanned pages but lose table structure. Layout-aware parsers such as Docling [**?**] recover reading order and tables, although they are not tuned to invoices. Multimodal transformers of the LayoutLM family [**?**, **?**] give the best results on forms and invoices but need labelled data and more compute.

Choice in this project. PDFs with a usable text layer are read directly with pdfplumber; scanned PDFs go through Docling; images are read by PaddleOCR after deskewing and limited upscaling. PaddleOCR's native engine aborted the host process on some platform and version combinations, so it runs in an isolated subprocess with a time limit rather than in the worker itself. Fields are then extracted with regular expressions. When the LayoutLMv3 weights are installed, the model reads every document a second time as an independent cross-check (a disagreement forces human review) and replaces the OCR reading of an image whose OCR quality falls below 0.5.

2.2.2 Document Classification

TF-IDF features with logistic regression are light, fast to train and easy to inspect, which suits a small number of document classes. Transformer classifiers such as BERT have a higher ceiling but need more data and compute than this task justifies. A weighted keyword classifier, which scores title phrases such as "tax invoice" or "purchase order", needs no training data but depends on the vocabulary it is given.

Choice in this project. A TF-IDF (character 2–5-grams) and logistic-regression classifier labels each document as an invoice, purchase order, quotation or *other* paperwork. It was trained and evaluated on a corpus of 1,260 generated documents that includes OCR-noise and missing-title variants, and was kept only because it beat the keyword classifier on validation data and on document kinds held out of training (Section 4.3). The keyword classifier remains the fallback whenever the model is less than 60% confident.

2.2.3 Vendor Deduplication and Identity Verification

Fuzzy name matching (rapidfuzz) tolerates spelling differences and corporate suffixes but can produce false matches on its own. Official identifiers are more reliable: an Indian GSTIN carries a check digit that can be validated offline, and an IFSC code can be looked up in a public bank directory, while live registry lookups are limited by API quotas.

Choice in this project. Extracted vendor names are normalised and matched with a fuzzy ratio threshold of 88. Vendor onboarding validates GSTIN format and check digit and looks IFSC codes up in a public directory, flags names 70–87% similar to an existing vendor as possible impostors, and never writes bank details directly: they become a pending change that a second finance or admin user must verify. Every upload is scanned with ClamAV, which is signature-based and therefore fast but offers no protection against zero-day threats.

2.2.4 Workflow Orchestration and Concurrency Control

Cron jobs and scripts are simple but not durable and handle deadlines poorly. Temporal offers durable timers, signals and replay, at the cost of a steeper learning curve. Redis or database locks stop two requests from claiming the same stock, but need careful failure handling. Kafka-compatible brokers [**?**] provide decoupled, replayable events between services, provided that consumers are idempotent; and a transactional outbox [**?**] ensures that an event is published if and only if the database change that caused it commits.

Choice in this project. Temporal runs the approval, reclaim and renewal workflows; a short-lived Redis lock guards each stock reservation; Redpanda is the event bus; and events are written through a transactional outbox and consumed with retries and a dead-letter table.

2.2.5 Usage Anomaly Detection and Explainability

A flat inactivity threshold is transparent but flags seasonal and weekend dips by mistake. Isolation Forest [**?**] is unsupervised and efficient on tabular data, though it is sensitive to its contamination setting and gives no explanation of its own; SHAP [**?**] supplies per-feature contributions at some extra cost.

Choice in this project. An Isolation Forest scores each licence from seven scale-free usage features (utilisation over 7, 30 and 90 days, user retention over 7 and 30 days, days since the last login and weekend share), and SHAP values explain which features pushed a licence towards "anomalous". The decision thresholds were chosen on a validation set, and a flagged licence becomes a reclaim request that the licence holder can contest during a seven-day grace period before it reaches an approver.

2.2.6 Contract Analysis and Clause Extraction

Keyword search misses paraphrased clauses. Legal-BERT [**?**] fine-tuned on CUAD [**?**] handles legal wording better and answers one question per clause, but it was trained mostly on US commercial contracts and is slow on a CPU. FinBERT [**?**] scores financial sentiment, which could serve as an adverse-media signal, although sentiment is only a rough proxy for risk.

Choice in this project. A publicly released Legal-BERT checkpoint fine-tuned on CUAD's training split [**?**] extracts the renewal term, the notice period and the expiration date; regular-expression rules are the fallback when the model is not installed or finds too little. The model was kept as the primary method because it beat the rules on CUAD's test split and read every value of the project's own templates correctly (Section 4.3). FinBERT was studied but not implemented.

2.2.7 Vendor Risk Scoring and Drift Monitoring

Static questionnaires are snapshots, whereas a learned classifier can combine several signals into one score. Random Forests [**?**] are robust on small tabular datasets and provide feature importances for explanation. Some facts, however, should not be weighed against others: a cancelled tax registration or a sanctions listing should decide the outcome on its own. The Population Stability Index (PSI) [**?**] compares the distribution of recent scores with the training distribution and signals model drift.

Choice in this project. Hard overrides are applied first (a cancelled or suspended GSTIN, or a potential sanctions match against the OpenSanctions consolidated list [**?**],

forces a High band); otherwise a Random Forest on six vendor features produces a probability that is mapped to Low, Medium or High. Missing signals are left out rather than counted against the vendor. A weekly PSI check flags drift but never retrains the model, so a person decides what to do with the signal.

2.2.8 Multi-Agent and Agentic Architectures

Multi-agent systems split a workflow among specialised components with clear ownership, at the cost of integration effort. Large-language-model (LLM) frameworks such as AutoGen [**?**] make agents cooperate through free-form conversation, which is flexible but hard to audit.

Choice in this project. In this report an *agent* is an independent service that observes events, decides with rules or trained models within the authority given to it by business rules, acts by changing its own data and publishing new events, and hands a decision to a person when its confidence is low or the action is sensitive. Coordination is through a shared event contract rather than conversation, which keeps every decision deterministic and auditable. An LLM is used in one place only: a conversational assistant that answers users' questions through read-only tool calls and makes no procurement decisions.

2.3 Related Works

Ten published works were examined, each covering one building block of the platform. They are grouped below by the problem they address and compared in Tables 2.2 and 2.3.

Document understanding. LayoutLM [**?**] was among the first models to learn from text together with its two-dimensional position on the page, and improved results on forms, receipts and invoices considerably. LayoutLMv3 [**?**] added image information through unified text and image masking and dropped the CNN backbone, giving a simpler and stronger model. On the tooling side, Docling [**?**] rebuilds reading order and table structure locally, and PaddleOCR 3.0 [**?**] provides multilingual OCR with structure analysis, although it runs best on a GPU.

Contract and risk language. CUAD [**?**] provides 510 expert-annotated contracts covering 41 clause categories and has become the standard benchmark for clause extraction. LEGAL-BERT [**?**] showed that pretraining on legal text clearly outperforms generic BERT on legal tasks. For the financial side of vendor risk, FinBERT [**?**] contributes sentiment analysis tuned to financial language.

Anomaly detection and explanation. Isolation Forest [**?**] finds unusual records by how quickly random splits isolate them, so it needs no labelled examples. SHAP [**?**] explains a single prediction by how much each feature pushed the score up or down.

Agent frameworks. AutoGen [**?**] represents the LLM-driven approach in which agents cooperate by conversation. It is included as a contrast to the event-driven design chosen here.

Taken together, the works show a clear pattern. The document-AI papers concentrate on how accurately fields can be read and say little about what should happen next, such as matching the vendor, catching duplicates or holding a suspicious bank-detail change

Table 2.2: Comparison of related works

Reference	Contribution	Advantages	Limitations	Role in this project
Xu et al., LayoutLM (KDD 2020) [?]	Joint pretraining of text and 2-D layout	Large gains on forms, receipts and invoices	No image features in pretraining; needs fine-tuning	Studied; basis of the LayoutLMv3 cross-check
Huang et al., LayoutLMv3 (ACM MM 2022) [?]	Unified text–image masking, no CNN backbone	Strong results; simpler architecture	Compute-heavy; needs labels for new document types	Implemented as an optional cross-check and image fallback
Auer et al., Docling (2024) [?]	Local layout, reading-order and table recovery	Fast, local, strong on tables	Not tuned to invoices; weak on poor scans	Implemented for scanned PDFs
Cui et al., PaddleOCR 3.0 (2025) [?]	OCR and structure-analysis toolkit	Accurate, multilingual	GPU preferred; native crashes on some platforms	Implemented for images, isolated in a subprocess
Hendrycks et al., CUAD (NeurIPS 2021) [?]	510 contracts, 41 clause categories	Realistic, expert-annotated benchmark	English, mostly US contracts	Training data of the clause model; its test split is used for evaluation

Table 2.3: Comparison of related works (continued)

Reference	Contribution	Advantages	Limitations	Role in this project
Chalkidis et al., LEGAL-BERT (EMNLP 2020) [?]	BERT pretrained on legal corpora	Beats generic BERT on legal tasks	Smaller corpus; mostly English	Implemented: CUAD-fine-tuned checkpoint extracts renewal clauses
Araci, FinBERT (2019) [?]	BERT further pretrained on financial text	Strong financial sentiment	Sentiment is only a proxy for risk	Studied; adverse-media signal left as future scope
Liu et al., Isolation Forest (ICDM 2008) [?]	Anomaly detection by random partitioning	Linear time, no labels needed	No native explanation; contamination must be set	Implemented for licence-usage anomalies
Lundberg and Lee, SHAP (NeurIPS 2017) [?]	Unified feature attribution	Consistent local explanations; fast for trees	Cost grows with features; not causal	Implemented: reasons shown for each flagged licence
Wu et al., AutoGen (2023) [?]	Multi-agent LLM conversation framework	Modular and customisable	Conversation-driven; hard to audit	Contrast: event-driven agents with durable workflows

for review; this platform builds a confidence-gated pipeline around the extraction step. The legal NLP work stops at extracting clauses, whereas here the extracted terms drive a renewal workflow and sit beside a live vendor risk score that is shown before anyone signs. Isolation Forest and SHAP are general-purpose tools; the contribution here is their use for licence reclaim with explanations and a grace period. Finally, frameworks such as AutoGen favour flexibility and free-form dialogue, whereas procurement needs predictable, auditable and concurrency-safe behaviour, which is why the project uses an event bus and durable workflows for orchestration.

2.4 Literature Summary and Research Gap

Table 2.4 summarises the techniques reviewed and where each is used in the proposed system.

Table 2.4: Techniques reviewed and their use in the proposed system

Area	**Best-known technique**	**Use in this system**
Layout and OCR	Docling, PaddleOCR, Tesseract	pdfplumber for text PDFs, Docling for scanned PDFs, PaddleOCR for images
Extraction cross-check	LayoutLMv3	Optional second reading; disagreement forces review
Classification	TF-IDF + logistic regression	Trained classifier with keyword fallback below 60% confidence
Clause extraction	Legal-BERT on CUAD	Primary extractor with regular-expression fallback
Licence analysis	Isolation Forest + SHAP	Anomaly score, explained reasons, reclaim after a grace period
Risk scoring	Classifier + rule overrides	Random Forest with GSTIN and sanctions overrides; PSI drift check
Orchestration	Temporal, locks, Kafka, outbox	Temporal workflows, Redis reservation lock, Redpanda with transactional outbox

What the literature mainly lacks is integration. Document extraction, clause analysis, anomaly detection and workflow orchestration are usually studied as separate problems and judged on benchmarks. Few systems join them into a single governed procurement pipeline, and fewer still do so with three properties in mind. The pipeline should be *fraud-aware*, with dual control of payment-detail changes and malware scanning that fails closed. It should be *accountable*, with an audit log of document, vendor, approval and contract actions, explanations for model outputs, and monitoring for model drift. And it should be *observable*, with per-service metrics, centralised logs that follow one correlation identifier across services, and a record of every event and notification. This project is an attempt to fill that gap.

Chapter 3

Proposed System

This chapter states the problem that motivated the project, defines what the work does and does not cover, and describes the proposed system: its architecture, agents, document pipeline, event flow, use cases and technology choices.

3.1 Problem Statement

In most organisations the IT procurement process is a chain of loosely connected manual steps. A purchase order or invoice arrives in whatever form the vendor chose, someone retypes its contents into a system, someone else checks the vendor, and an approval request travels by e-mail until somebody replies. Contracts are filed with their renewal dates noted on a calendar, and software licences are renewed without checking whether anyone still uses them. Each step is slow on its own, and together they leave room for costly mistakes and for fraud. Table 3.1 lists the main problems and their impact.

Table 3.1: Problems in the current procurement process and their impact

Problem	**Impact**
Documents arrive through disconnected channels in inconsistent formats	Manual data entry, delays and extraction errors
Vendor records are not de-duplicated and payment-detail changes are accepted on request	Duplicate vendors, BEC fraud and payments to the wrong account
Approvals are routed by hand with no SLA or stock checks	Stalled requests, overspending and two requests consuming the same stock
Software licence usage is not monitored, or is judged by a flat inactivity rule	Money spent on unused seats, or seats reclaimed by mistake
Contract terms are tracked informally	Silent auto-renewals and missed notice periods
Vendor risk is assessed once, at onboarding	Stale risk ratings
No single view of a request across tools	Difficult auditing and slow investigation of problems

These problems are connected. A vendor record that was never checked is exactly what makes a fraudulent bank-detail change possible, and an extraction error that goes unnoticed can become a duplicate payment, so fixing one step in isolation helps only a little. The problem addressed in this project is therefore: *to design and build an automated, event-driven procurement platform that reads unstructured procurement documents, links every document to a vendor record, routes approvals under spend and deadline controls, manages contracts and renewals, and keeps a vendor risk score up to date, while keeping automated decisions explainable and auditable and keeping a person in the loop for low-confidence results and sensitive changes.*

3.2 Scope

The project covers the path from an incoming procurement document or purchase request to a signed contract, the receipt of goods and the matching of the vendor's invoice. Within this scope the system scans and reads documents, matches vendors, routes approvals with spend-tier and SLA controls, reserves inventory, monitors licence usage, generates, signs and tracks contracts, scores vendor risk, and keeps users informed through e-mail notifications, all behind role-based access control and with metrics, logs and an audit trail.

The work does not include payment execution, ERP integration, adverse-media screening, live GSTIN registry lookups as a default (the client exists but is switched off), a third-party e-signature provider in the demonstrated configuration (Documenso and DocuSign clients exist but signing is done in the application), automatic retraining of models, or a production or Kubernetes deployment. All data is synthetic: the vendor risk training set, the licence usage logs, the classifier corpus and the sample documents are generated, and the CUAD contracts are used only to evaluate clause extraction.

3.3 Scope Design

The system design defines the structure, components, interfaces and information flow of the platform. Its scope covers the following areas:

- **System architecture:** the user interface, gateway, agent services, event bus, data stores and their relationships.
- **User interface design:** role-specific pages through which users submit requests, review extracted data, approve or reject requests, sign contracts and inspect vendors, licences and notifications.
- **Agent coordination:** how the agents cooperate through events, for example from document intake to purchase request, approval, contract and notification.
- **Data flow:** how information moves from user input through processing and analysis to stored records and messages.
- **External dependencies:** ClamAV signature updates, an SMTP service (Mailpit in development), the HaveIBeenPwned password range API, a public IFSC directory, the Frankfurter exchange-rate API, the OpenSanctions consolidated list, Hugging Face model downloads, and a local Ollama model for the assistant.
- **Output and decision support:** confidence scores, risk bands with contributing factors, anomaly reasons for licences, and the risk shown to a signer before signing.

The system assists procurement decisions and does not guarantee correct extraction in every case, which is why low-confidence results are sent to human review. This scope is the basis for the architecture, interface, component and data-flow descriptions in this chapter and the next.

3.4 Proposed System

3.4.1 Overview and Architecture

The proposed system is an event-driven platform of four cooperating agents. Each agent is one or more independent FastAPI services with their own data and responsibilities, and the agents communicate mainly through events on a Kafka-compatible bus (Redpanda) rather than by calling one another. Users reach the system through a React frontend and an Nginx gateway; every service verifies the caller's JWT and role itself. The services share one PostgreSQL database, in which each service owns and migrates its own tables, together with Redis for locks and caching and MinIO for document files. Figure 3.1 shows the overall structure.

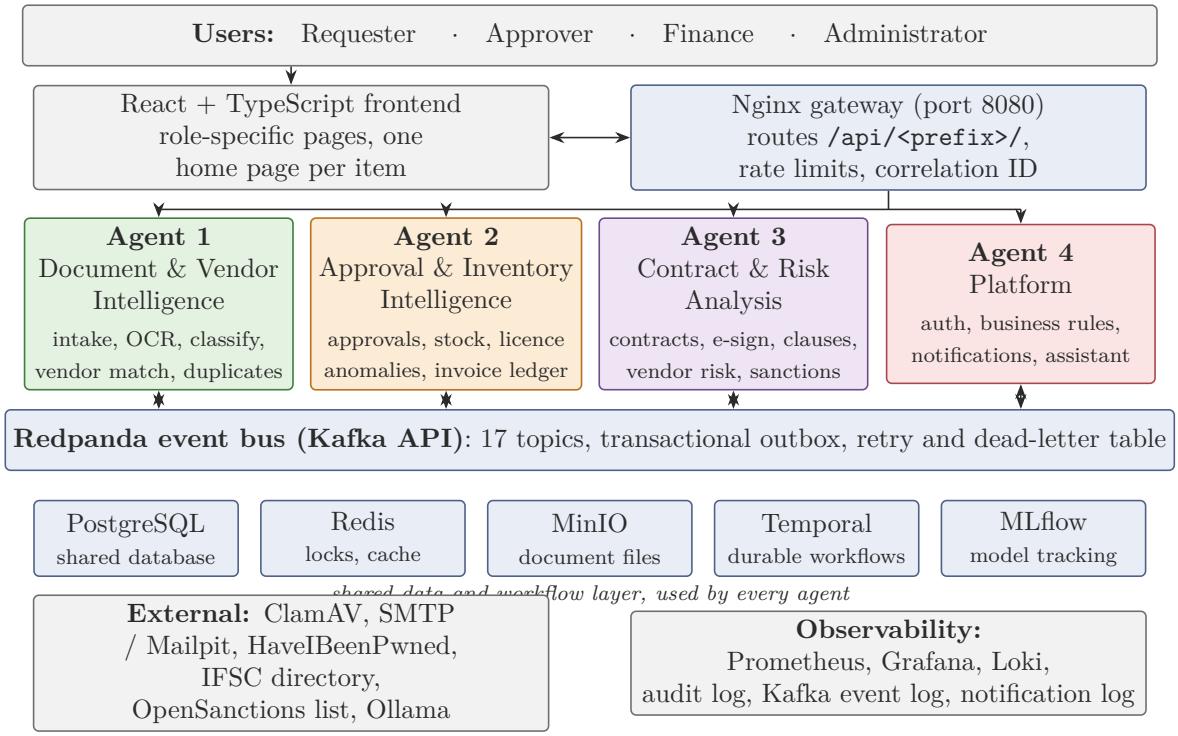


Figure 3.1: Overall architecture of the proposed system

Splitting the system this way keeps each concern separate. A change to the extraction logic does not touch the approval engine, and a slow document parse cannot block a user's upload, because heavy work runs in background workers off the request path.

3.4.2 The Four Agents

Table 3.2 summarises the agents, and the paragraphs that follow describe what each one does. In the sense defined in Section 2.2.8, each agent observes events, decides within the limits set by the business rules, acts, and defers to a person when its confidence is low or the action is sensitive.

Document and Vendor Intelligence. This agent is the entry point for every document, whether uploaded, e-mailed to an intake address or dropped into a watched folder. It checks the file's size and real type, streams it through ClamAV and rejects it if the scanner flags it or cannot be reached, stores it in MinIO and publishes `document.ingested` in the same database transaction. A background worker reads, classifies and extracts the document, gives every field a confidence score and sends doubtful results to a review queue. The agent matches the vendor against existing records, books invoices against their purchase request, flags likely duplicates, and never applies new bank details directly: they wait as a pending change until a different user with the finance or admin role verifies them, and payment is held meanwhile.

Approval and Inventory Intelligence. This agent derives each request's approval chain from spend tiers held in the business-rules engine (automatic up to Rs. 500, a department manager up to Rs. 5,000, and a manager plus the finance head above that) and from an approval-authority matrix that also enforces separation of duties: nobody, including an administrator, can approve their own request. A Temporal workflow per

Table 3.2: Summary of the four agents in the proposed system

Agent	Port	Main responsibility	Key techniques
Document and Vendor Intelligence	8001 (+ worker)	Document intake, reading, classification, field extraction, vendor matching and onboarding, duplicate detection, bank-detail control	ClamAV, pdfplumber, Docling, PaddleOCR, TF-IDF + logistic regression, LayoutLMv3, rapidfuzz
Approval and Inventory Intelligence	8002 (+ worker)	Approval routing and authority, SLA escalation, inventory reservation and reorder, licence anomalies and reclaim, invoice ledger	Temporal, Redis locks, Isolation Forest, SHAP
Contract and Risk Analysis	8003 (+ worker)	Contract generation and signing, clause extraction, renewal reminders, vendor risk scoring, sanctions screening, offboarding	Jinja2, Legal-BERT (CUAD), Random Forest, MLflow, PSI
Platform	8004, 8005, chatbot	Authentication and roles, business-rules engine, notifications, conversational assistant	JWT, argon2id, Jinja2, SMTP, Ollama (Llama 3.2 3B)

request waits for the decision, reminds the approver at 50% and 80% of the SLA and escalates after 48 hours. Stock is reserved under a short Redis lock so that two concurrent requests cannot take the last unit, and a request for an item that is out of stock is created as back-ordered. An Isolation Forest scores licence usage; a licence above the anomaly threshold receives a reclaim request that waits through a seven-day grace period before it reaches an approver. When `contract.signed` arrives, the request moves to *ordered*; a user then marks the goods as received (*fulfilled*), and invoices are matched line by line against the order, with a 1% tolerance on the unit price and on the order's remaining balance before an invoice is held.

Contract and Risk Analysis. When a request is approved, this agent fills one of three Jinja2 templates (hardware purchase, SaaS subscription or professional services), renders the contract as text and PDF, and sends it for signature automatically. Signing happens in the application: an authorised approver, finance or admin user types or draws a signature, gives consent, and the contract and signer details are sealed with a SHA-256 hash recorded in the audit log and shown on a signature certificate. If the vendor is rated High or has not been scored, the signer must explicitly acknowledge the risk. An HMAC-verified, replay-protected webhook accepts signatures from external providers and is used by the tests. A Legal-BERT model reads the renewal term, notice period and expiration date from the contract text, and a Temporal workflow sends renewal reminders 60, 30 and 15 days before the notice deadline. The agent also scores each vendor's risk, screens vendor names against a sanctions list, runs a weekly PSI drift check, and handles vendor offboarding by flagging active contracts for review instead of closing them.

Platform. The fourth agent owns what no functional agent should. The authentication service issues short-lived JWT access tokens and refresh tokens held in an httpOnly cookie, hashes passwords with argon2id, verifies e-mail addresses, locks an account after repeated failures and rejects passwords found in known breaches. It also hosts the business-rules

engine: 47 rules, such as spend tiers, thresholds and tolerances, that an administrator edits at run time and that services refresh when `business_rule.updated` is published. The notification service consumes 11 event topics, sends urgent messages immediately and batches routine ones into digests, retries failed deliveries and logs every message. Finally, a conversational assistant answers questions about requests, licences and vendors using a local Llama 3.2 (3B) model through Ollama; it can only call six read-only tools, each executed with the asking user's own token, so it sees nothing the user could not see.

3.4.3 Document Processing Pipeline

The most involved part of the system turns an uploaded file into a trustworthy structured record (Figure 3.2). The upload returns as soon as the file is stored, and the worker processes it in the background.

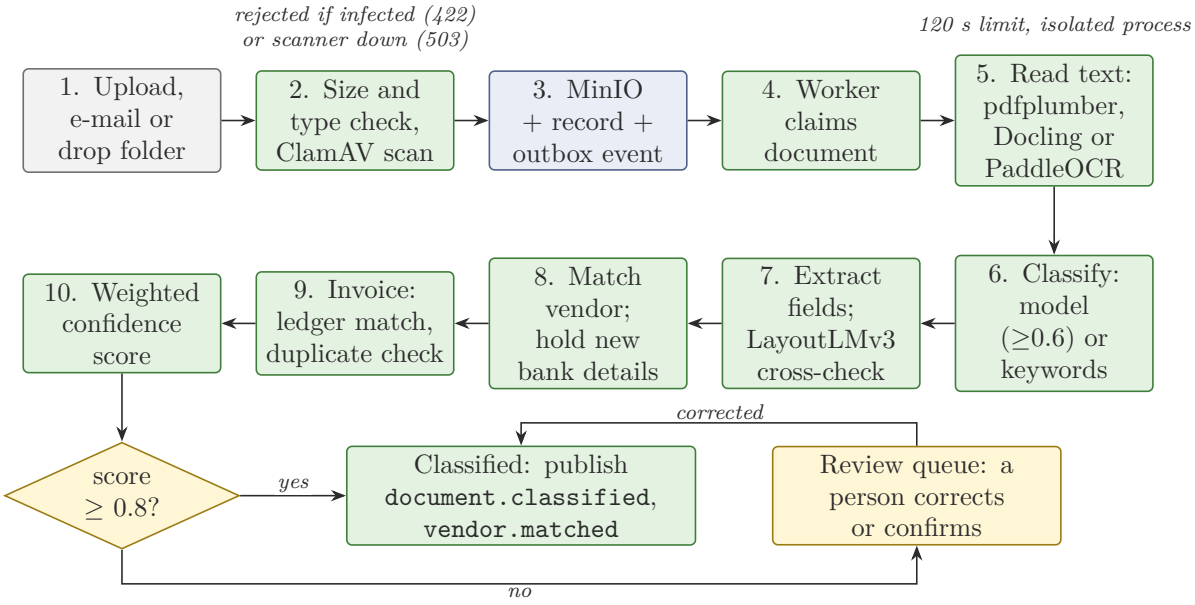


Figure 3.2: Document processing pipeline of the Document and Vendor Intelligence agent

A file is rejected before storage if it is empty, too large (25 MB at the gateway), of a type that its bytes do not confirm, infected, or if ClamAV cannot be reached, so the intake fails closed. After the file and its database row are stored, the worker reads it in a separate process with a 120-second limit: pdfplumber for PDFs with a usable text layer, Docling for scanned PDFs, and PaddleOCR for images after deskewing and upscaling by at most 1.5 times. The classifier labels the document as an invoice, purchase order, quotation or other paperwork, and regular expressions extract the vendor name, document number, date, currency, totals and line items. When LayoutLMv3 is installed it reads the document independently, and a disagreement forces review. The vendor name is normalised and matched against existing vendors with a fuzzy ratio of at least 88, or a new vendor record is created; first-seen bank details are held for verification. An invoice is booked against its purchase request and checked for duplicates: same vendor, an amount within 2%, a date within 7 days and a document number at least 85% similar (all three values are business rules). Finally, the per-field confidence scores are combined into an overall score (Section 4.3); a document scoring below 0.8 goes to the review queue, and reviewers' corrections are kept as per-vendor hints for later documents. Any dependency failure returns the document to the queue to be retried, up to five attempts.

3.4.4 Inter-Agent Communication and Workflow

Every event uses the same envelope (event ID, event type, timestamp, source service, correlation ID and payload), and the 17 topics and their payloads are fixed in a shared contract document so that producers and consumers could be built in parallel. Each service authenticates to the broker with its own SASL/SCRAM account and may use only its own topics. Events are written to an outbox table in the same transaction as the change that caused them and relayed to the broker afterwards; consumers are idempotent, retry failures and move events that keep failing to a dead-letter table shown on the Control Panel. Figure 3.3 shows the main flows and Table 3.3 lists the topics.

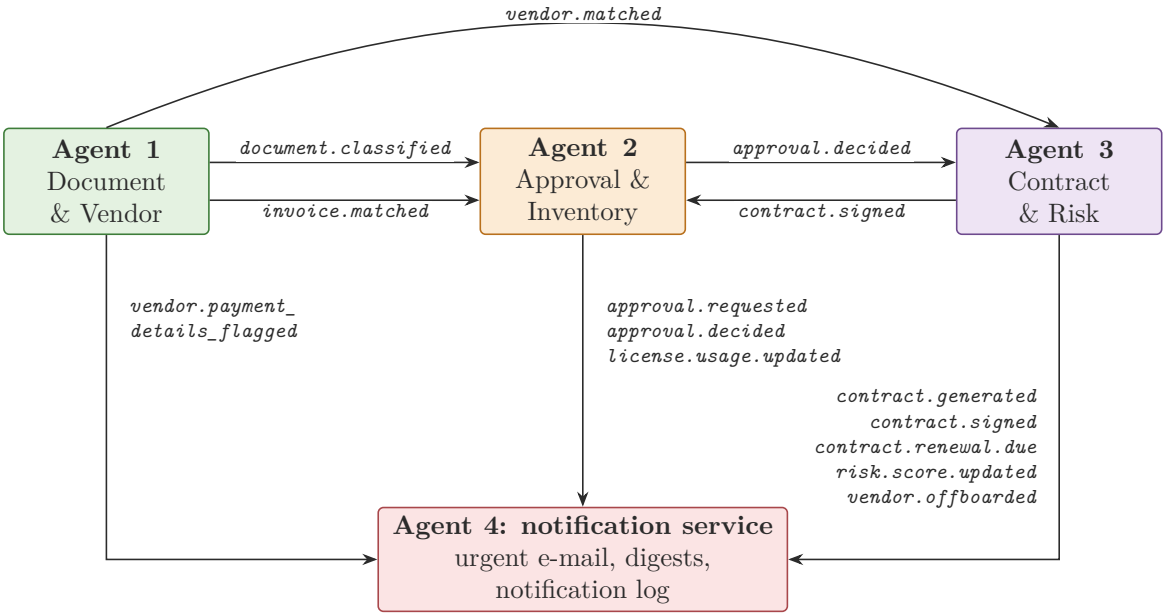


Figure 3.3: Main event flows between the agents. Rule changes (`business_rule.updated`) are broadcast to every agent and are omitted for clarity.

Table 3.3: Event topics exchanged between the agents

Topic	Published by	Consumed by and purpose
`document.ingested`	Agent 1	Agent 1 worker: starts the pipeline
`document.classified`	Agent 1	Agent 2: turns a quote into a purchase request; notifications
`vendor.matched`	Agent 1	Agent 3: scores the vendor's risk
`vendor.payment_details_flagged`	Agent 1	Notifications: urgent alert on a pending bank-detail change
`invoice.matched`	Agent 1	Agent 2: updates the invoice ledger and request status
`approval.requested`	Agent 2	Notifications: tells the approver
`approval.decided`	Agent 2	Agent 3: generates the contract; Agent 1: books invoices waiting for the request; notifications
`license.usage.updated`	Agent 2	Notifications: utilisation and anomaly alerts

Table 3.3 (continued)

Topic	Published by	Consumed by and purpose
`order.summary.generated`	Agent 2	Control Panel: hourly open-order summary
`contract.generated`	Agent 3	Notifications
`contract.signed`	Agent 3	Agent 2: request becomes *ordered*; notifications
`contract.renewal.due`	Agent 3	Notifications: 60, 30 and 15-day reminders (urgent at 15)
`risk.score.updated`	Agent 3	Notifications: urgent for the High band
`vendor.offboarded`	Agent 3	Notifications: offboarding summary and contracts flagged
`reminder.due`	Agents 1–3	Control Panel; e-mail delivery is future work
`business_rule.updated`	Agent 4	Agents 1–3: refresh their cached rules
`notification.send`	Any service	Notifications: generic message

In a typical run, a requester creates a purchase request, or a vendor quote is dropped into the intake folder and Agent 1 classifies it and Agent 2 creates the request from it. Agent 2 builds the approval chain and, once the last approver agrees, publishes `approval.decided`. Agent 3 generates the contract and sends it for signature, and its `contract.signed` event moves the request to *ordered*. The notification service follows all of these events, sending urgent alerts such as an approval request or a flagged bank-detail change immediately and batching routine ones into a digest. Every API request receives a correlation ID at the gateway, which is carried through the events and log lines of every service, so the whole journey of one request can be followed in Grafana.

3.4.5 Use Case View

Figure 3.4 presents the use cases of the four roles. The *Requester* raises purchase requests, uploads documents and follows their progress; the *Approver* decides the requests and licence reclaims assigned to them in the approval-authority matrix and can sign contracts; the *Finance* user verifies bank-detail changes, onboards vendors, maintains risk signals and can also approve and sign; and the *Administrator* manages users and business rules, monitors the system and offboards vendors. Administrators can perform every action, so the diagram shows each role only with the actions specific to it.

The use cases group into five phases:

1. **Authentication and business rules.** Users sign up, verify their e-mail and sign in; an administrator assigns roles and edits the business rules (spend tiers, SLA, thresholds and tolerances) on the Business Rules page, with every change kept in a history.

2. **Purchase and inventory.** A requester submits a purchase request, stock is checked and reserved, the approval workflow runs in Temporal, and the approver approves or rejects.

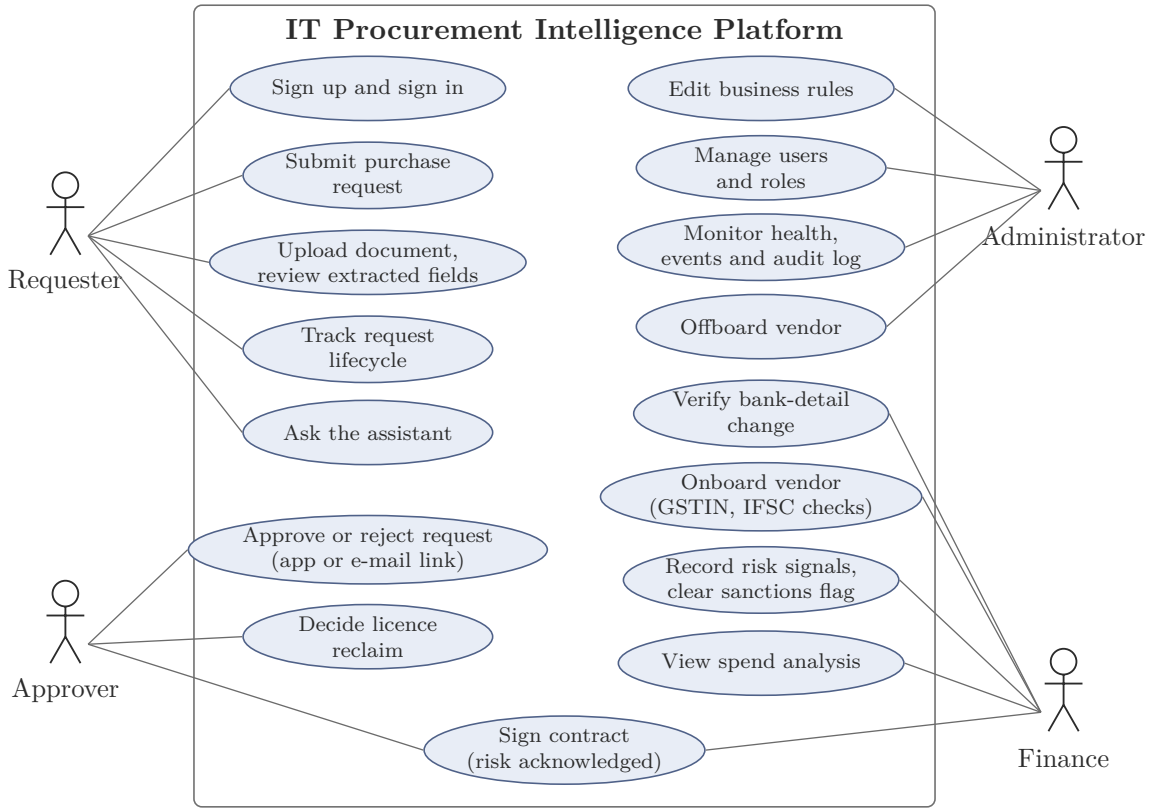


Figure 3.4: Use case diagram of the IT procurement platform

3. **Vendor intelligence.** Vendor documents are read and matched, new vendors are onboarded with identifier checks, bank details are verified by a second person, and a vendor risk score is calculated.

4. **Contract and risk.** A contract is generated for the approved request, sent for signature, signed in the application after the vendor's risk is shown, and renewal reminders are scheduled.

5. **Notification.** Approvals, flagged bank details, renewals, risk changes and signatures trigger e-mail notifications to the people concerned.

3.4.6 Technology Stack

Table 3.4 lists the main technologies. Pretrained open-source components are used wherever possible; the models trained within the project are the document classifier, the licence-anomaly model and the vendor risk model.

3.4.7 Key Design Principles

Four principles run through the design. The system is *fraud-aware*: uploads are scanned and rejected if the scanner is unavailable, bank details need a second person's verification, the webhook is accepted only with a valid HMAC signature and never twice, and signing a contract with a High-risk vendor requires an explicit acknowledgement. It is *human-centred*: results below the confidence threshold go to review, a licence reclaim waits for the holder and an approver, offboarding flags contracts rather than closing them, and the models never retrain themselves. It is *accountable*: significant actions are written

Table 3.4: Technology stack of the proposed system

Layer	**Technology**	**Purpose**
Frontend	React, TypeScript, Vite, Tailwind CSS	Role-based pages, one home page per item
Gateway	Nginx	Single entry point; routes `/api/<prefix>/`, rate limits, security headers
Backend services	FastAPI (Python), SQLAlchemy, Alembic, Docker Compose	One containerised service per agent, plus workers
Messaging	Redpanda (Kafka API), aiokafka, SASL/SCRAM	Events between agents, per-service topic permissions
Workflow engine	Temporal	Approval SLAs, reclaim grace period, renewal reminders
Data stores	PostgreSQL, Redis, MinIO	Records, locks and caches, document files
Document AI	pdfplumber, Docling, PaddleOCR, LayoutLMv3, rapidfuzz	Text extraction, OCR, cross-check, vendor and duplicate matching
Models	scikit-learn (logistic regression, Isolation Forest, Random Forest) [**?**], SHAP, Legal-BERT (Hugging Face Transformers), MLflow	Classification, licence anomalies, vendor risk, clause extraction, experiment tracking
Assistant	Ollama with Llama 3.2 (3B), tool calling	Read-only questions about requests, licences and vendors
Security	JWT, argon2id [**?**], HMAC-SHA256, ClamAV, HaveIBeenPwned	Authentication, password hashing, webhook verification, malware scanning, breached-password check
Monitoring	Prometheus, Grafana, Loki, Promtail, kafka-exporter	Metrics, dashboards, centralised logs by correlation ID, consumer lag
Notifications	Jinja2 templates, SMTP (Mailpit in development)	Templated e-mail

to an audit log, identity is always taken from the JWT and never from a request body, business-rule changes are versioned, and every model-routing decision is logged. Finally, it is *observable*: every service exposes Prometheus metrics, logs carry a correlation ID, the Control Panel shows every Kafka event and every failed event, and a System Health page shows the state of the services.

Chapter 4

Design of the System

This chapter describes the requirements, development approach, system models, resource requirements and architecture of the platform. It presents the functional and non-functional requirements, the data flows, the main UML views, the database design and the system layers.

4.1 Requirement Engineering

Requirement engineering identifies the capabilities expected from the platform, documents them, and keeps them consistent during development. It mattered particularly in this project because the system is divided into services that must interoperate through shared event formats, database tables, REST conventions and authentication rules.

4.1.1 Requirement Elicitation

Requirements were identified from the following sources:

- **Existing practices and literature:** the review in Chapter 2 and the problem analysis in Chapter 3 identified the recurring problems: repeated manual data entry, unverified vendor details, duplicate invoices, delayed approvals, under-used licences, missed renewals and infrequent vendor-risk reviews.
- **Stakeholder analysis:** the users are requesters, approvers, finance users and system administrators, implemented as the four roles *requester*, *approver*, *finance* and *admin*.
- **Guide and mentor reviews:** discussions with the guides prioritised features and separated what could be built from what depends on paid accounts or providers. A structured mentor review on 29 August 2026 produced a list of required changes that was tracked item by item to completion.
- **Review of tools and datasets:** document-processing tools, workflow engines and public datasets were tried against their expected inputs and practical limits. For example, PaddleOCR's native engine crashed the worker process on some platform and version combinations, which led to running it in an isolated subprocess rather than dropping it; and the CUAD dataset was chosen to evaluate clause extraction.

The requirements were then grouped into functional and non-functional requirements.

4.1.2 Software Lifecycle Model

The project followed an iterative and incremental model (Figure 4.1). This suited a system whose thresholds, approval rules, models and integration details had to be tested and refined during implementation. Each team member owned one service. The main practices were:

- **Contract-first design:** the Kafka event envelope and topic payloads, the base database schema, the REST response envelope and the authentication middleware were agreed before the services were built in parallel.
- **Service-wise increments:** the work was divided across the document and vendor, approval and inventory, contract and risk, and notification and authentication services.

- **Integration iterations:** the services were connected behind one gateway. Integration exposed problems in gateway routing, broker addressing, UUID column types and colliding migration tables, which were fixed at the shared level.
- **Continuous testing and audits:** unit and end-to-end tests were written alongside the features, and two audit passes (5 September and 17 October 2026) re-tested the running system, fixed the issues found and added a regression test for each fix.
- **Repeatable deployment:** a single `run.sh` script builds and starts the whole containerised stack from a clean clone.

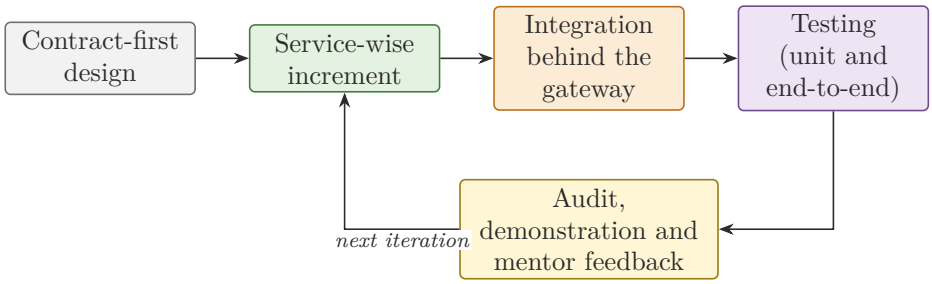


Figure 4.1: Iterative and incremental development cycle

4.1.3 Requirement Analysis

The requirements are grouped into functional requirements, which describe what the system does, and non-functional requirements, which describe the qualities and constraints it must satisfy. Every threshold mentioned below is a business rule that an administrator can change at run time; the values given are the current settings.

4.1.3.1 Functional Requirements

Table 4.1: Functional requirements

ID	Requirement	Agent
FR1	Users can sign up, confirm their e-mail address, sign in and receive an access token and a refresh token. Every endpoint checks the token and the role, except the health and metrics endpoints and the HMAC-authenticated signature webhooks.	Platform
FR2	An administrator can view and edit the business rules (spend tiers, SLA, thresholds and tolerances); each change is versioned and reaches every service without a restart.	Platform
FR3	Documents can be uploaded, e-mailed to an intake address or dropped into a watched folder. Each file is size- and type-checked and scanned for malware before storage; it is rejected if the scanner is unavailable.	Document and Vendor
FR4	Documents are classified as invoice, purchase order, quotation or other; vendor, document number, date, currency, totals and line items are extracted.	Document and Vendor
FR5	Each extracted field receives a confidence score. Documents below the threshold (0.8) go to a review queue, where fields can be corrected or confirmed.	Document and Vendor

Table 4.1 (continued)

ID	Requirement	Agent
FR6	Vendors are matched by normalised name (fuzzy threshold 88); onboarding validates GSTIN format and check digit and IFSC codes and flags possible impostors.	Document and Vendor
FR7	New or changed bank details stay pending, with payment held, until verified by a different user with the finance or admin role.	Document and Vendor
FR8	Likely duplicate invoices are flagged (same vendor, amount within 2%, date within 7 days, document number at least 85% similar).	Document and Vendor
FR9	Purchase requests follow approval chains by spend tier (automatic up to Rs. 500, manager up to Rs. 5,000, manager and finance above) and an approval-authority matrix; nobody can approve their own request.	Approval and Inventory
FR10	Each request's approval runs as a durable Temporal workflow with reminders at 50% and 80% of the SLA; after 48 hours the request escalates to the next approver, and a timeout at the last level rejects it. Approvers can also decide from a single-use e-mail link for amounts up to Rs. 50,000.	Approval and Inventory
FR11	Stock is reserved under a Redis lock so that concurrent requests cannot reserve the same unit; a request for out-of-stock items is back-ordered, and reorder suggestions are computed.	Approval and Inventory
FR12	Licence usage is scored by an anomaly-detection model with per-licence explanations; an anomalous licence receives a reclaim request that waits through a seven-day grace period before approval.	Approval and Inventory
FR13	Invoices are matched line by line against the purchase request, and an invoice exceeding the price or remaining balance by more than 1% is held.	Approval and Inventory
FR14	Contracts are generated from templates after approval and sent for signature automatically; renewal type, notice period and end date are extracted, and reminders are scheduled 60, 30 and 15 days before the notice deadline.	Contract and Risk
FR15	Contracts are signed in the application with consent and a sealed audit record; signing for a High-risk or unscored vendor requires an acknowledgement. External signature callbacks are accepted only with a valid HMAC signature and only once.	Contract and Risk
FR16	Vendor risk is a score and a Low, Medium or High band with its contributing factors; a cancelled GSTIN or a potential sanctions match forces High until finance clears it with a reason.	Contract and Risk
FR17	The risk model is checked weekly for drift with the Population Stability Index; it is never retrained automatically.	Contract and Risk
FR18	Vendor offboarding revokes the vendor's access, flags active contracts for review and sets a data-retention flag instead of deleting records.	Contract and Risk
FR19	The notification service consumes events, sends templated e-mail, sends urgent items immediately and batches routine ones into digests, retries failures and keeps a searchable log.	Platform
FR20	An assistant answers questions about requests, licences and vendors using only read-only tools and the asking user's permissions.	Platform

Table 4.1 (continued)

ID	Requirement	Agent
FR21	The frontend provides role-specific navigation and dashboards, a request wizard, an approval inbox, one home page per request, document, vendor, licence and contract, and pages for notifications, spend analysis, business rules, users, audit activity, system health and the Kafka event log.	Frontend
FR22	Significant actions on documents, vendors, requests and contracts are written to an audit log that administrators can query.	All

4.1.3.2 Non-Functional Requirements

Table 4.2: Non-functional requirements

Category	Requirement
Security	JWT authentication and role checks in every service; identity taken only from the token; argon2id password hashing and breached-password check; account lockout and gateway rate limits; fail-closed malware scanning; HMAC-verified webhooks; dual control of payment details; per-service Kafka credentials and topic permissions; only the gateway port exposed.
Reliability	Approval, reclaim and renewal workflows are durable (Temporal). Events are published through a transactional outbox; consumers are idempotent, retry and dead-letter failures; a sweep re-queues stranded documents; a timed-out last approval is rejected, never approved by default.
Consistency	A common REST envelope and error format; a purchase-request state machine enforced in code and by a database constraint; idempotent migrations owned by each service.
Performance	Uploads return once the file is stored; reading, OCR and model inference run in background workers with time limits.
Traceability	Every event carries an event ID, source and correlation ID; logs from every service are collected centrally and searchable by correlation ID; significant actions are audited; model-routing decisions are logged.
Explainability	Confidence scores per field, risk factors per vendor, SHAP reasons per licence and the risk shown before signing.
Maintainability	Responsibilities separated by service; shared contracts documented; thresholds held as business rules, each with a behaviour test.
Portability	The stack is containerised and runs on macOS, Linux and Windows through WSL2.

Constraints and assumptions. Payment execution, ERP integration, adverse-media screening, a third-party e-signature provider and automatic model retraining are outside the scope. Kafka traffic is authenticated but not encrypted, which is acceptable only on a single host. The vendor risk data, licence usage logs, classifier corpus and sample documents are synthetic.

4.1.4 UML Diagrams and Data Flow Diagrams

4.1.4.1 Use Case Diagram

The use case diagram is shown in Figure 3.4 in Chapter 3. It identifies the Requester, Approver, Finance user and Administrator and groups their activities into authentication and rules, purchase and inventory, vendor intelligence, contract and risk, and notification.

4.1.4.2 Data Flow Diagram – Level 0

The context diagram (Figure 4.2) shows the platform as a single process and its exchanges with the external entities: the three groups of users, vendors who send documents, the e-mail service, and the external lookup services.

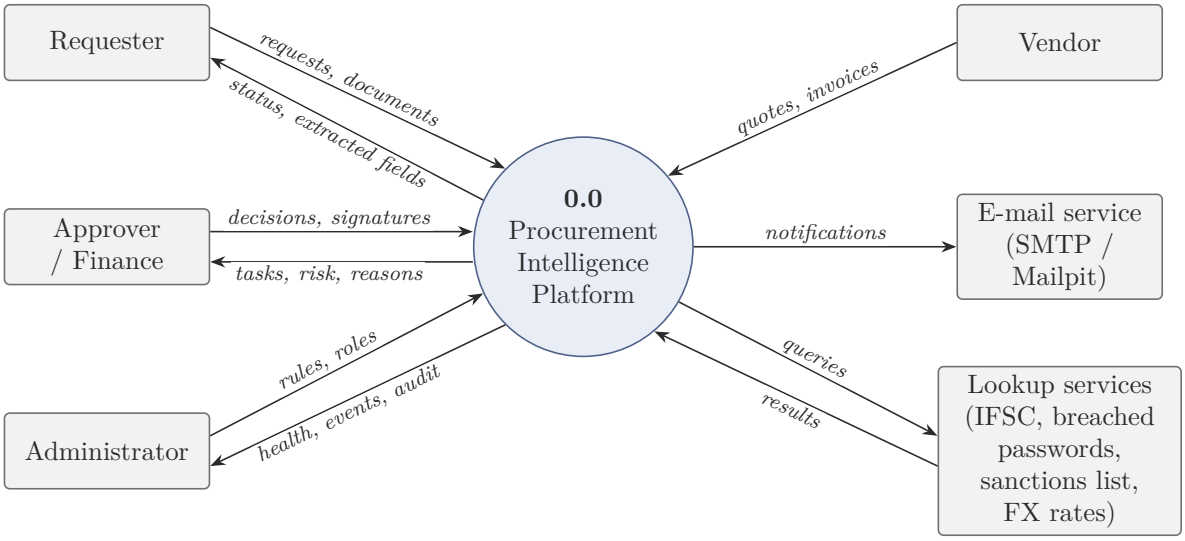


Figure 4.2: Level 0 data flow diagram (context diagram)

4.1.4.3 Data Flow Diagram – Level 1

The Level 1 diagram (Figure 4.3) decomposes the platform into the processes of Table 4.3. Processes 2.0 to 5.0 exchange events through the messaging layer, process 6.0 consumes them, and process 1.0 issues the tokens and rules that the others check.

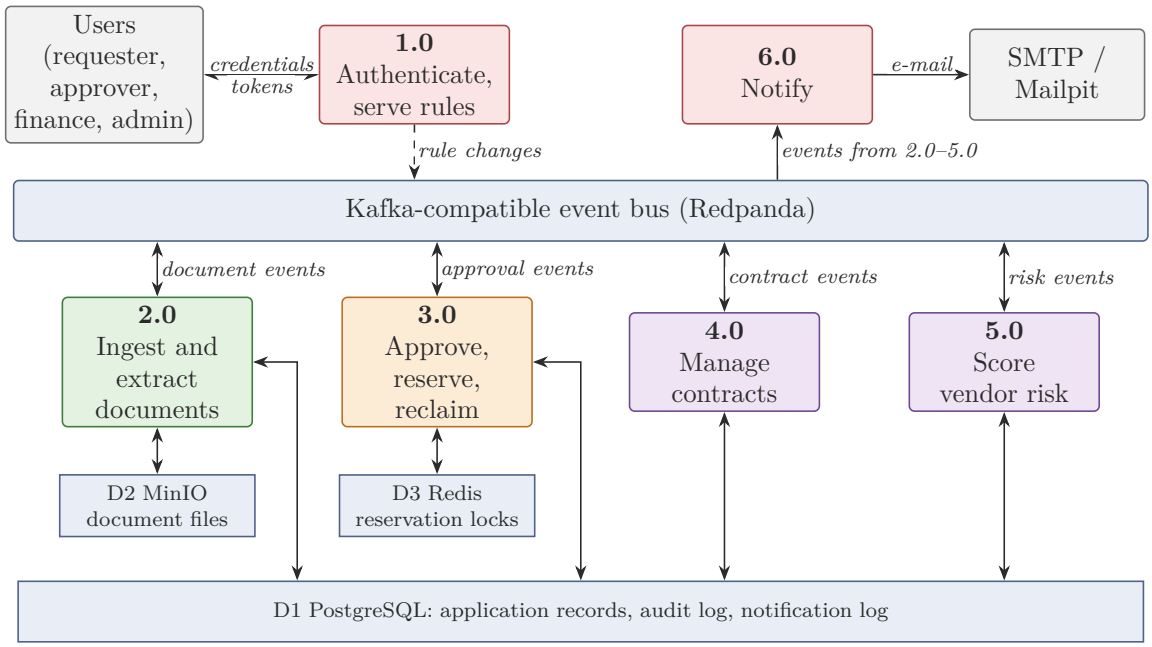


Figure 4.3: Level 1 data flow diagram

Table 4.3: Level 1 DFD processes

Process	Name	Inputs	Outputs
1.0	Authenticate, serve rules	Credentials, refresh token, rule edits	Access and refresh tokens, role, rule changes
2.0	Ingest and extract documents	Uploaded, e-mailed or dropped file	Classified document, vendor match, review-queue items, invoice match
3.0	Approve, reserve, reclaim	Purchase request, decisions, usage logs	Decisions, reservations, escalations, reclaim requests
4.0	Manage contracts	Approved request, signature	Contract text and PDF, signature record, renewal reminders
5.0	Score vendor risk	Vendor signals, sanctions list	Risk score, band, factors, overrides, drift flag
6.0	Notify	Events from 2.0–5.0	E-mail, digests, notification log

4.1.4.4 Sequence Diagram

Figure 4.4 follows one purchase request from creation to signature. It shows the synchronous calls through the gateway, the asynchronous events, and the point at which the Temporal workflow hands the decision back to the service.

4.1.4.5 Activity Diagram: Document Processing

The activity diagram (Figure 4.5) shows the decisions taken for one document. The upload is rejected if the file fails a check, is infected or cannot be scanned; a file that cannot be read fails with a message for the uploader. A readable file is classified, extracted, matched and scored. A document meeting the threshold, with no rule forcing review, is completed; otherwise it waits in the review queue until a reviewer corrects or confirms it.

4.1.4.6 Purchase-Request State Machine

A purchase request moves through the states of Figure 4.6. The allowed transitions are defined once in a shared module and checked on every status change, and the database rejects any status outside the list. An event that arrives after the request has already moved past the state it would cause (for example a late `contract.signed`) is recognised and ignored rather than treated as an error.

4.1.4.7 Entity Relationship Diagram

All services share one PostgreSQL database of about fifty tables; each service creates and migrates its own. Figure 4.7 shows the main entities and their foreign keys as they exist in the running database. The vendor is the central entity: documents, purchase requests, contracts, licences, risk scores, sanctions screenings and payment-change requests all refer to it. The `inventory` table has no foreign key: requests refer to stock by SKU inside their items. Supporting tables, not shown, hold users and tokens, business rules and their history, the event outbox, inbox and dead-letter table, the Kafka event log, notification and model-routing logs, and the reviewer-feedback tables.

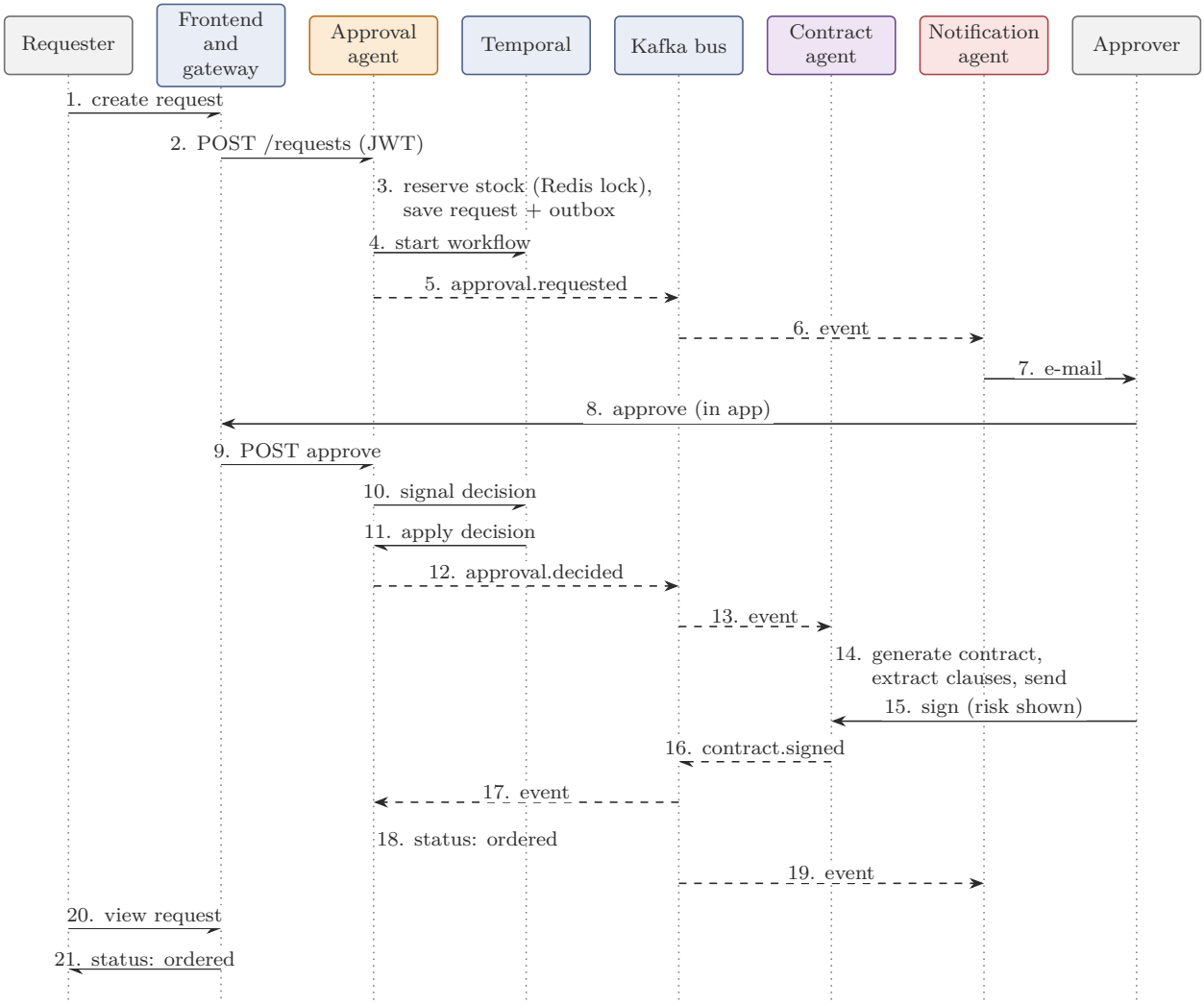


Figure 4.4: Sequence diagram of the purchase-request journey (solid: synchronous call; dashed: asynchronous event)

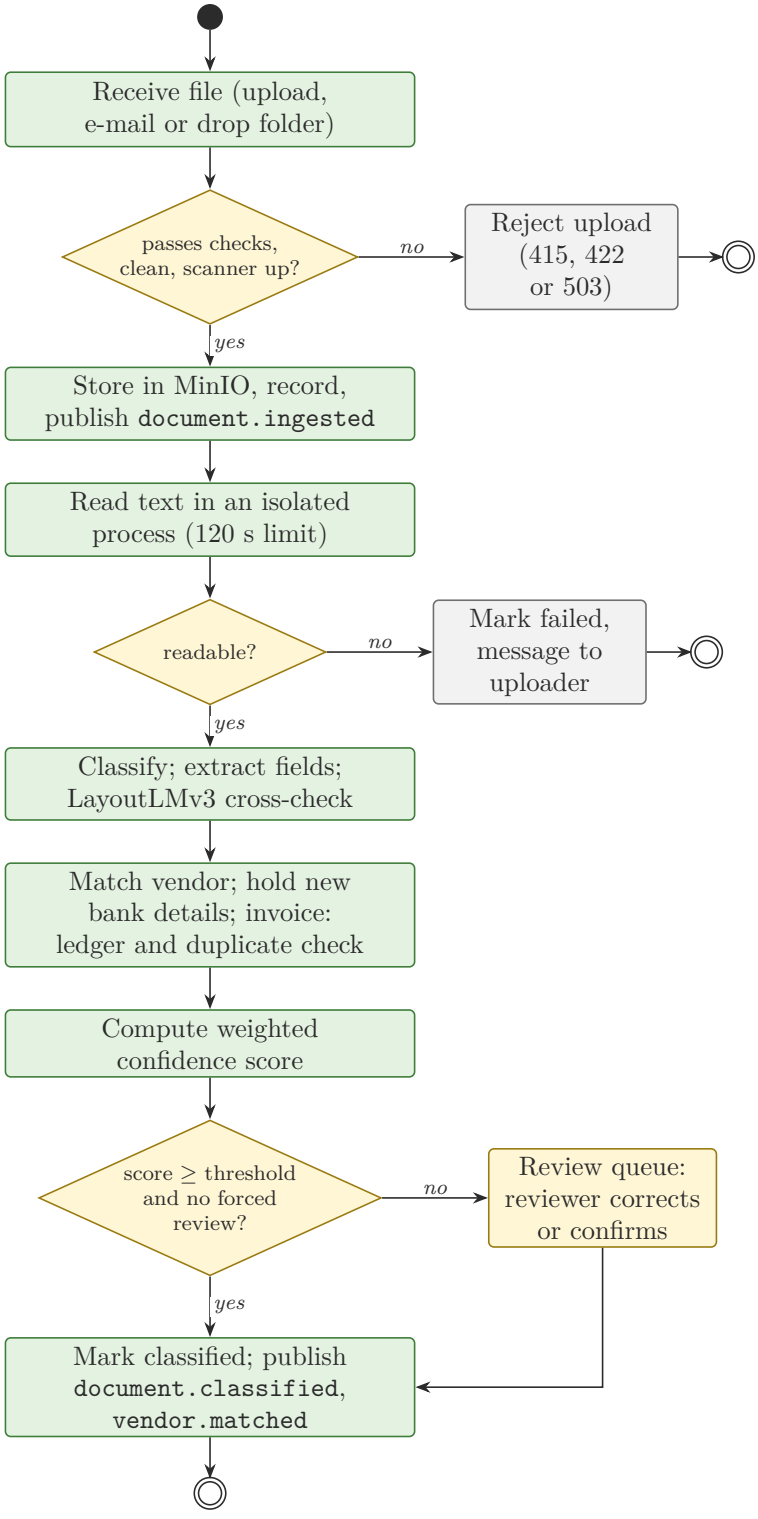


Figure 4.5: Activity diagram of document processing

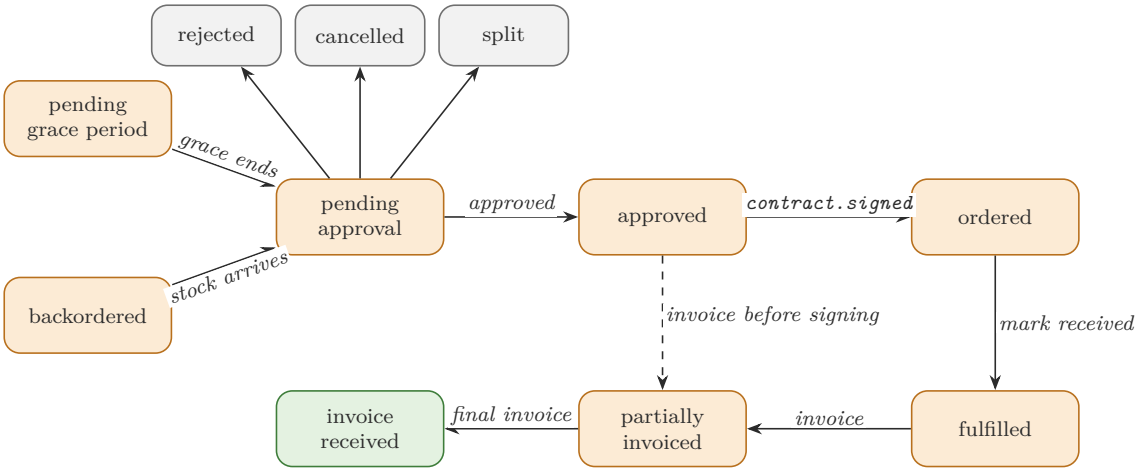


Figure 4.6: State machine of a purchase request (main transitions; stored state names use underscores, for example `pending_approval`; a request can also be cancelled from the waiting states)

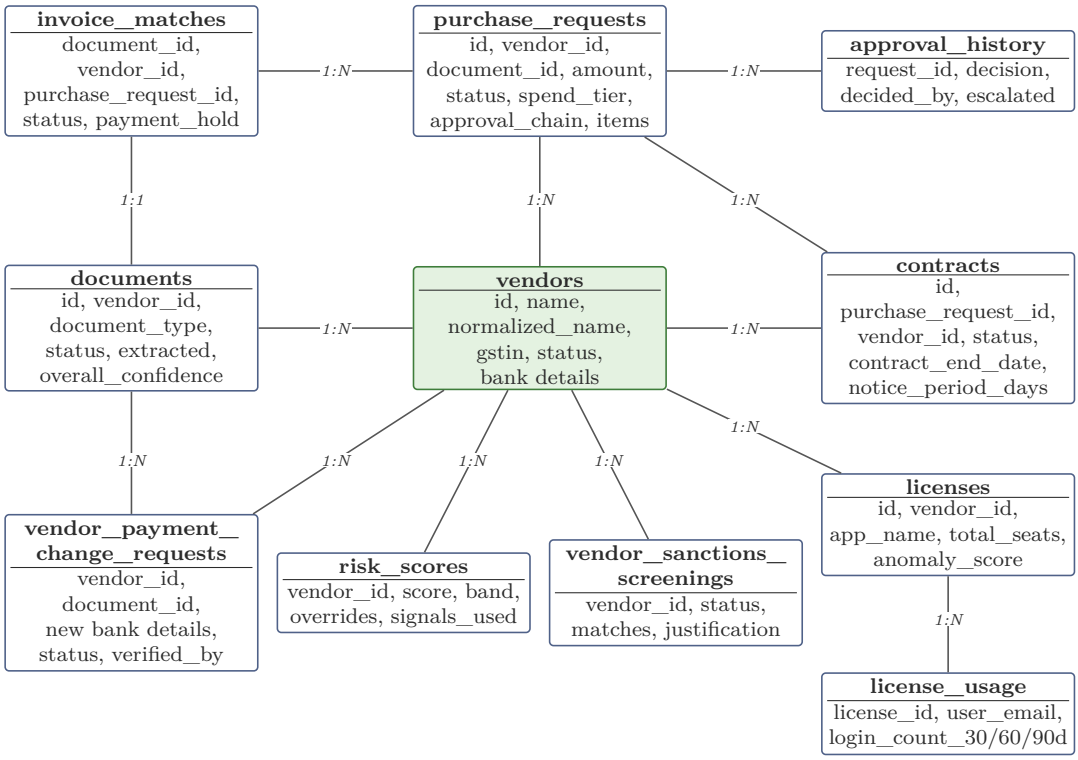


Figure 4.7: Entity relationship diagram of the main entities (foreign keys as in the running database; `invoice_matches` also refers to `vendors`)

4.1.5 Cost Analysis

The project uses only open-source software and free API tiers, so it incurs no software-licence cost, and it was developed on the team's own laptops. The potential costs are team effort, paid API tiers if live registry lookups are enabled, cloud hosting, and a production e-mail service. The estimates in Table 4.4 are indicative and should be revised against real quotations.

Table 4.4: Indicative cost analysis

Item	Basis	Indicative cost
Software licences	Python, FastAPI, PostgreSQL, Redis, Redpanda, MinIO, Temporal, MLflow, Prometheus, Grafana, Loki, ClamAV, Docling, PaddleOCR, Hugging Face models, Ollama, React and Docker are open source	Rs. 0
Development hardware	Existing student laptops	Rs. 0 additional
External APIs	HaveIBeenPwned range API, IFSC directory and Frankfurter exchange rates are free; the OpenSanctions list is free for non-commercial use only (CC BY-NC 4.0) and needs a licence for commercial use; live GSTIN lookups depend on the provider's quota	Rs. 0 in the project; depends on usage and licensing in production
Team effort	Part-time project work by four students	Not monetised
Optional cloud hosting	One virtual machine with about 4–8 vCPU and 16 GB RAM, plus storage and backups	Approx. Rs. 6,000–10,000 per month
E-mail delivery	Production SMTP service instead of the development mail catcher	A few hundred rupees per month at pilot volume

A meaningful break-even estimate needs measured document volumes, licence costs and the time saved or duplicate payments avoided. Those figures are not available, so no return-on-investment claim is made.

4.1.6 Hardware and Software Requirements

The stack runs 24 containers on one Docker host. Measured on 16 October 2026, it used about 3.2 GB of RAM when idle and more while reading scanned documents, and its images occupied about 15 GB before build cache and data volumes. ClamAV alone needs about 1 GB of memory. Table 4.5 gives the resulting requirements and Table 4.6 the software.

4.2 System Architecture

The platform follows a modular, event-driven architecture in which independent services each own one procurement function. The frontend talks to the services through the gateway with synchronous HTTP requests where an immediate answer is needed, while the services talk to one another mainly through events. Temporal runs the long-lived workflows; PostgreSQL holds the records, Redis the locks and caches, and MinIO the files. Background workers consume events and run slow tasks, so users get a response without waiting for every downstream step.

Table 4.5: Hardware requirements

Component	**Minimum**	**Recommended**
Processor	4 cores	8 cores
Memory	8 GB available to Docker	16 GB RAM
Storage	40 GB free	80 GB free for images, models (about 1 GB), sanctions data and volumes
Network	Required for the first build, model and sanctions downloads, and sign-up's breached-password check	Broadband connection
GPU	Not required	Optional, for faster OCR and transformer inference

Table 4.6: Software requirements

Category	**Requirement**
Operating system	macOS, Ubuntu/Linux, or Windows 10/11 with WSL2
Container runtime	Docker Engine or Docker Desktop with Compose v2
Backend	Python 3.11, FastAPI, SQLAlchemy, Alembic, pytest, scikit-learn, SHAP, Transformers, rapidfuzz, Jinja2
Frontend	React, TypeScript, Vite and Tailwind CSS, built in a Node 20 container
Data and messaging	PostgreSQL, Redis, Redpanda (Kafka API) and MinIO
Workflow and ML	Temporal, MLflow
Document processing	pdfplumber, Docling, PaddleOCR, LayoutLMv3 (optional download), ClamAV
Monitoring and mail	Prometheus, Grafana, Loki, Promtail, kafka-exporter, Mailpit
Assistant	Ollama on the host with the Llama 3.2 (3B) model
Gateway and browser	Nginx; a current Chrome, Edge, Firefox or Safari

4.2.1 UI/UX Design

The frontend is a React and TypeScript single-page application built with Vite and Tailwind CSS. Nginx serves it and routes its API calls, so the page and the API share one origin. The interface follows these principles:

1. **Role-aware navigation:** each role sees its own navigation and dashboard. Frontend route checks are only for usability; every endpoint enforces authorisation itself.
2. **One home page per item:** a request, document, vendor, licence and contract each have one page where all their actions live; lists, dashboards and the approval inbox link to it. A request page shows a clickable lifecycle whose steps open their details and actions.
3. **Clear asynchronous status:** decisions and document processing complete in background workers, so the interface polls briefly after an action and shows each step as it settles; a step is shown as in progress only while an agent is actually working on it.
4. **Human review:** extracted fields are shown with their confidence and can be corrected or confirmed; risk factors, licence anomaly reasons and the vendor's risk at signing are always visible.
5. **Honest limits:** where a feature has no backend support the interface shows it as unavailable rather than pretending it succeeded.
6. **Demonstration support:** the sign-in page offers one-click demo accounts in development builds, and a command palette (Ctrl/Cmd+K) searches loaded requests, vendors, contracts and documents.

Screenshots of the implemented screens are given in Section **??**, and Table 4.7 lists the principal screens.

4.2.2 Block Diagram

The block diagram (Figure 4.8) shows the deployed components and their connections on the single Docker host. Table 4.8 describes each layer.

4.2.3 Main Data Paths

The principal data paths are:

1. **User access:** the browser loads the frontend from Nginx; API calls are routed to the right service after the gateway strips the `/api/<prefix>/` segment.
2. **Document processing:** the Document and Vendor agent checks and scans an upload, stores it in MinIO and publishes `document.ingested` through the outbox. Its worker reads, classifies and extracts the document and publishes `document.classified` and `vendor.matched`.
3. **Approval processing:** the Approval and Inventory agent reserves stock, saves the request and starts a Temporal workflow. An approve or reject call signals the workflow; the worker applies the decision, updates the status and publishes `approval.decided`.

Table 4.7: Principal screens and their purposes

Screen	User role	Purpose
Sign in, sign up, password reset	All	Account access and e-mail verification
Dashboards	Each role	Own requests, items awaiting approval, spend overview or system overview
New request wizard	Requester	Enter details, choose a vendor, optionally upload a document with live pipeline status, review extracted fields and submit
Request page	All, by permission	Clickable lifecycle, approval, procurement checks, invoice ledger, contract and signature
Approval inbox	Approver, finance	Pending requests and reclaims for the levels assigned to the user, with SLA status
Document page	Requester, approver, finance, admin	Preview, extracted fields with confidence, correction and confirmation
Vendor page and onboarding	Finance, admin	Risk score and factors, risk signals, bank-detail verification, sanctions status, onboarding checks
Licence page	Approver, finance, admin	Usage trend, anomaly score and reasons, open reclaim request
Contract page	Approver, finance, admin	Contract text, renewal dates, signing dialog, signed copy and certificate
Inventory, spend analysis, risk	Finance, admin	Stock and reorder suggestions, spend by category, vendor risk bands and drift
Business rules, users	Admin	Edit rules with history; manage roles
Control Panel, Controls, System Health, Audit	Admin (Controls: finance too)	Kafka event log and failed events, platform controls, service and model-routing status, audit trail
Notifications, account	All	Notification history; password change and sign-out everywhere

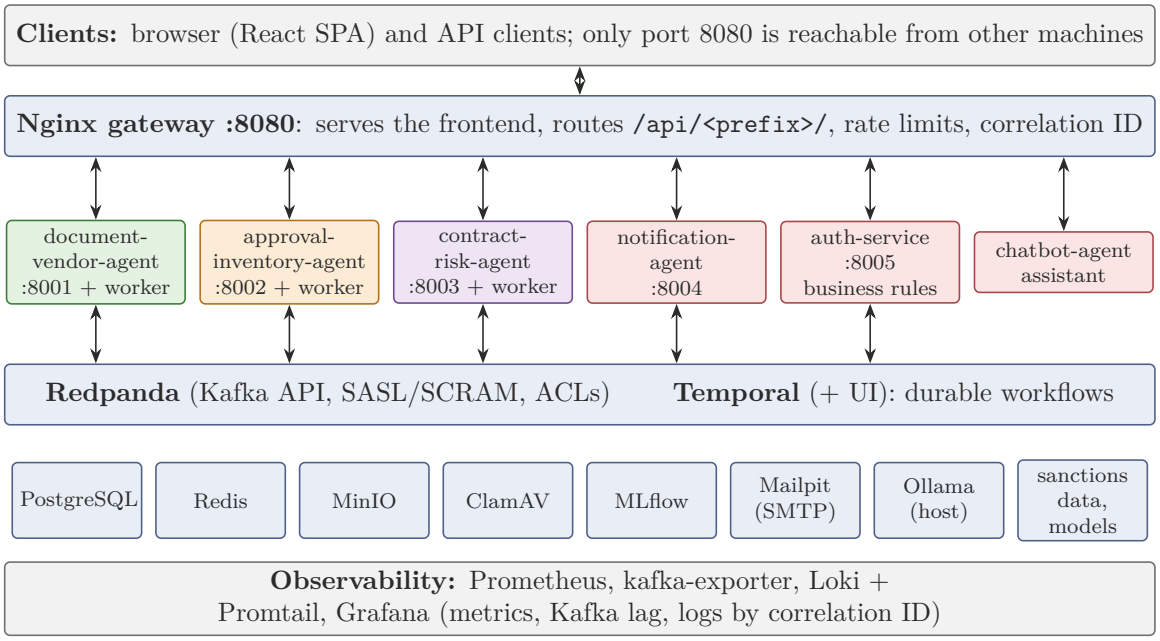


Figure 4.8: Block diagram of the deployed system

Table 4.8: Architectural layers

Layer	Components and responsibilities
Clients	The React application and API clients reach the platform only through the gateway on port 8080.
Gateway	Nginx serves the frontend, routes API paths to the services, limits request rates on authentication endpoints and assigns a correlation ID.
Application services	Document and Vendor agent (8001), Approval and Inventory agent (8002), Contract and Risk agent (8003), notification agent (8004), authentication and business-rules service (8005) and the assistant. Every service checks the JWT and role, except health, metrics and the HMAC-signed webhooks.
Background workers	The document, approval and contract services each run a worker that consumes events, runs Temporal workflows and performs slow tasks such as OCR and model inference.
Messaging and orchestration	Redpanda carries the events with per-service credentials; Temporal runs the approval, reclaim and renewal workflows.
Data and platform services	PostgreSQL holds the records, Redis the reservation locks and rule caches, and MinIO the files. ClamAV scans uploads, MLflow tracks the risk model and drift, Mailpit receives development e-mail, and Ollama runs the assistant's model on the host.
Observability	Prometheus collects metrics, Loki collects logs, and Grafana shows request rates, latency, errors, Kafka consumer lag and the logs of one correlation ID across services.

4. **Contract processing:** the Contract and Risk agent generates the contract for an approved request and sends it for signature. When it is signed, `contract.signed` moves the request to *ordered*; marking the goods as received makes it *fulfilled*, and the vendor's invoice is matched against it.

5. **Risk assessment and notifications:** the Contract and Risk agent publishes risk, renewal and offboarding events; the notification agent consumes them and sends e-mail.

6. **Supporting services:** PostgreSQL stores the records, Redis guards reservations, MinIO stores files, and MLflow records the risk model's training runs and drift checks.

Together, these paths show how synchronous API calls, asynchronous events and durable workflows combine to carry a request from intake to completion.

4.3 x

Bibliography

[1] X.

Acknowledgement

The success of a project like this, which demands technical expertise, patience and generous support from guides, is possible only when team members work together. We take this opportunity to express our gratitude to those who were instrumental in its completion. We thank our project guide **Dr. Jyoti More** and our external guide **Mr. Alok** for their tremendous support and help; without them, this project would not have reached fruition. We also thank our project coordinator **Dr. Rakhi Kalantri** for regular inputs on documentation and the project timeline, and our Head of Department **Dr. M. Kiruthika** for her encouragement. Finally, we thank our Principal, **Dr. S. M. Khot**, and our college, **Fr. C. Rodrigues Institute of Technology, Vashi**, for the opportunity and the environment to learn and grow.

Project Group Members:

1. More Anjali Dinesh, 1023207

_____

2. Sankaye Vaidehi Jagdeep, 1023239

_____

3. Sawant Niraj Ravindra, 1023240

_____

4. Shete Anooj, 1023250

_____

Appendix B

The project team has identified the following Sustainable Development Goals (SDGs) as relevant to the project. The mapping shows the relationship between the project problem, the proposed solution and each selected SDG.

SDG Mapping of the Project

Sr. No.	SDG No.	Complete Name of SDG	Rationale / Contribution of the Project
1	9	Industry, Innovation and Infrastructure	The platform applies machine learning, event-driven software architecture and workflow automation to a core enterprise process, improving the technological capability and efficiency of IT procurement.
2	12	Responsible Consumption and Production	Duplicate-invoice detection, inventory reservation and reorder suggestions, and the reclaim of under-used software licences reduce wasted purchases and idle resources.
3	16	Peace, Justice and Strong Institutions	Role-based access, separation of duties in approvals, dual control of payment-detail changes, sanctions screening, audit logging and explainable decisions support accountable, transparent and fraud-resistant institutions.

Guidelines for Students:

- Identify only the SDG(s) that have a direct and meaningful connection with the project.
- Mention the official SDG number and complete name.
- Provide a clear rationale explaining how the project contributes to the selected SDG.
- Avoid mapping an SDG merely because the project falls within a particular technology domain.
- The SDG mapping should be discussed and validated with the Project Guide.

_____ Name of Project Guide: Dr. Jyoti More

Signature of Project Guide

Date: _____

Appendix C

Appendix C: Complex Engineering Problem Identification and Evaluation

The Washington Accord emphasises the ability of engineering graduates to identify, analyse and solve complex engineering problems using appropriate engineering knowledge, modern tools, multidisciplinary thinking and analytical reasoning. The Major Project provides an opportunity for students to demonstrate these abilities through a structured and evidence-based project.

This appendix provides a structured mechanism to assess the complexity of the Major Project. The assessment is first carried out by the student team and then verified by the Project Guide.

A. Project Details

Project Title	Agentic AI Procurement Solution
Group / Team No.	
Name of Student(s)	More Anjali Dinesh (1023207), Sankaye Vaidehi Jagdeep (1023239), Sawant Niraj Ravindra (1023240), Shete Anooj (1023250)
Project Guide	Dr. Jyoti More
Academic Year	2026–2027

B. Complexity Evaluation

For each parameter, the student team assigns a score from **0 to 3** with a justification or evidence. The Project Guide reviews the assessment and records the verified score in the final column.

Sr. No.	Complexity Parameter	Student Score	Student Justification / Evidence	Guide Score
1	Problem novelty and research depth	2	Integrates document AI, workflows and risk scoring in one governed pipeline; the individual techniques are established (Chapter 2).	
2	Multidisciplinary knowledge requirement	3	Combines document processing, machine learning, natural language processing, distributed systems, workflow engines, security and web development.	
3	System design complexity	3	Six FastAPI services and three workers in 24 running containers, a shared event and database contract, a transactional outbox, durable workflows and a gateway.	
4	Technology integration	3	FastAPI, Redpanda, Temporal, PostgreSQL, Redis, MinIO, ClamAV, MLflow, Prometheus, Grafana, Loki, Ollama and React in one containerised stack.	
5	Data analysis and experimentation	2	Three models trained and kept only after beating a baseline on held-out data; Legal-BERT evaluated on 102 CUAD test contracts; extraction measured on 15 PDFs and 24 image variants. All training data is synthetic (Section 4.3).	
6	Societal / industrial relevance	2	Addresses duplicate payments, unused licences and payment-fraud risks that affect organisations.	
7	Stakeholder involvement	2	Four user roles modelled; guidance from an external industry guide and mentor feedback rounds.	
8	Sustainability considerations	1	Reduces wasted purchases and unused licences.	
	Total Score	**18 / 24**		**/ 24**

Scoring Guideline: 0 – Not applicable / negligible; 1 – Basic level of complexity; 2 – Moderate level of complexity; 3 – Significant / high level of complexity. Maximum score: 24.

C. Final Project Classification

The final classification is based on the score verified by the Project Guide.

Score Range	Project Classification
0–8	Simple Engineering Problem
9–16	Moderately Complex Engineering Problem
17–24	Complex Engineering Problem

Final Classification:

☐ Simple Engineering Problem ☐ Moderately Complex Engineering Problem ☐ Complex Engineering Problem

Date: _____

Project Guide Signature